

Pharmacy Assessment

Credentials

maheenimran84.9b9b72a38b68@agentforce.com

PWD: Royal7861us@

1. Apex Classes

Class Name	Purpose
CartAPI_abc	REST endpoint to create a shopping cart for a customer.
ProductAPI_abc	REST endpoint to retrieve all pharmacy products.
AddCartItemAPI_abc	REST endpoint to add products into an active shopping cart.
CheckoutAPI_abc	REST endpoint to checkout an active shopping cart and create an order.
ProductService_abc	Business logic for retrieving available products.
CartService_abc	Handles shopping cart creation and retrieval.
CheckoutService_abc	Implements checkout business logic including order creation, stock validation, inventory update, and cart completion.
ApiResponseUtil_abc	Utility methods for creating standardized API responses.
OrderController_abc	Controller for the Visualforce warehouse order page.

2. DTO Classes

DTO	Purpose
ApiResponse_abc	Standard API response wrapper.
ProductDTO_abc	Product response model.
CartItemResponseDTO_abc	Shopping cart item response.
CartResponseDTO_abc	Shopping cart response model.

CreateCartRequestDTO_abc	Request payload for creating a cart.
AddCartItemRequestDTO_abc	Request payload for adding a product into a cart.
CheckoutRequestDTO_abc	Request payload for checkout.
OrderResponseDTO_abc	Checkout response containing created order details.

3. Batch & Scheduler Classes

Class	Purpose
CloseAbandonedCartsBatch_abc	Batch Apex that closes shopping carts that have remained Active for more than seven days.
CloseAbandonedCartsScheduler_abc	Schedules the batch job to execute automatically every day.

4. Sample Data Loader

Class	Purpose
SampleDataLoader_abc	Creates sample customers, products, shopping carts, cart items, and orders for testing and demonstration purposes.

5. Visualforce Components

Component	Purpose
OrderPage_abc	Warehouse order management page displaying all customer orders with expandable order items.

6. Custom Objects Used and Their Purpose

Custom Object	Purpose
Customer_abc__c	Stores customer information such as customer name and unique customer code. This object serves as the primary entity for identifying customers when creating shopping carts and placing orders through the REST APIs.
Product_abc__c	Stores the pharmacy inventory, including product name, unit price, available stock quantity, category, and expiry information. This object is used by the Product API to display available medicines and by the Checkout process to validate stock availability and update inventory after a successful purchase.
Shopping_Cart_abc__c	Represents a customer's active shopping session. It groups multiple cart items together before checkout and maintains the cart lifecycle using statuses such as Active, Checked Out, and Abandoned. This object enables customers to build their orders before completing the purchase.
Shopping_Cart_Item_abc__c	Stores the individual products added to a shopping cart, including product, quantity, and unit price. It acts as the child object of Shopping_Cart_abc__c and allows a single cart to contain multiple products. During checkout, these records are converted into Order Items.
Order_abc__c	Represents the finalized purchase after a successful checkout. It stores order-level information such as customer, order number, order status, order date, and effective date. This object is also displayed on the warehouse Visualforce page for order tracking and management.
Order_Item_abc__c	Stores the individual products purchased within an order. It contains product, quantity, and unit price information and is a child object of Order_abc__c. Separating order items from the order allows one order to contain multiple products while maintaining a normalized data model.

Why a Separate Cart and Order?

The assessment required implementing a complete purchasing workflow. Therefore, the design separates the shopping phase from the completed purchase phase.

- Shopping Cart represents products the customer intends to purchase.
- Shopping Cart Items store products before checkout.
- Order represents a completed transaction.
- Order Items preserve the purchased products even after the shopping cart has been checked out.

This separation reflects how e-commerce applications typically manage customer purchases, ensuring historical orders remain unchanged while customers can continue creating new shopping carts for future purchases.

Reason for Using Custom Objects Instead of Standard Objects

Custom objects were used to build a simplified Pharmacy Management System tailored to the assessment requirements. Standard Salesforce objects such as Account, Contact, Product2, and Order have built-in dependencies (e.g., Price Books, Opportunities, Contracts) that were not required for this solution. Using custom objects provided complete flexibility to design the data model, implement the required business logic, and keep the application simple, self-contained, and focused on the assessment objectives.

7. REST API Endpoints

Method	Endpoint	Description
GET	/services/apexrest/abc/products	Retrieve all available pharmacy products.
POST	/services/apexrest/abc/cart	Create an active shopping cart.
POST	/services/apexrest/abc/cart/items	Add a product into the shopping cart.

POST `/services/apexrest/abc/cart/checkout` Checkout the shopping cart and create an order.

Salesforce Features Used and Their Purpose

Salesforce Feature	Purpose in the Assessment
Custom Objects	Used to model the pharmacy management system by creating business entities such as Customer, Product, Shopping Cart, Shopping Cart Item, Order, and Order Item.
Custom Fields	Used to store business-specific information such as Customer Code, Product Stock Quantity, Unit Price, Order Status, Cart Status, Effective Date, and Order Date.
Lookup Relationships	Used to associate Customers with Shopping Carts and Orders, and Products with Shopping Cart Items and Order Items while maintaining flexible relationships between records.
Master-Detail Relationships	Used between Order and Order Item to establish a parent-child relationship, enabling automatic ownership inheritance, cascade deletion, and roll-up summary support if required.
Apex Classes	Used to implement the application's business logic including shopping cart management, inventory validation, checkout processing, and REST service implementation.
Apex REST Services	Used to expose RESTful APIs that allow external applications such as Postman to create carts, add products, retrieve products, and perform checkout operations.
Wrapper Classes (DTOs)	Used to create clean request and response models for REST APIs, ensuring standardized JSON payloads and reducing direct dependency on Salesforce objects.
Batch Apex	Used to automatically process shopping carts that remained active for more than seven days and update their status to Abandoned, demonstrating asynchronous processing of large data volumes.
Schedulable Apex	Used to schedule the Batch Apex job so abandoned cart processing runs automatically at predefined intervals without manual intervention.

Visualforce	Used to build a warehouse management page that displays customer orders and allows warehouse staff to expand each order and view purchased products.
Visualforce Controller	Used to retrieve order information from Salesforce, prepare data for display, and implement expand/collapse functionality within the Visualforce page.
SOQL (Salesforce Object Query Language)	Used extensively to retrieve Customers, Products, Shopping Carts, Cart Items, Orders, and Order Items while supporting relationship queries and filtering.
DML Operations (Insert, Update)	Used to create and update Salesforce records during shopping cart creation, product addition, order creation, stock updates, and cart checkout.
Exception Handling (Try-Catch)	Used to gracefully handle business validation failures such as customer not found, empty cart, or insufficient stock while returning meaningful API responses.
Transactions & Savepoints	Used during checkout to ensure that if any operation fails (such as stock validation or order creation), the complete transaction is rolled back, maintaining data consistency.
JSON Serialization & Deserialization	Used to convert Apex objects into JSON responses and convert incoming JSON request payloads into Apex DTO objects for REST API communication.
HTTP Annotations (@HttpGet, @HttpPost)	Used to define REST API methods that respond to HTTP GET and POST requests according to RESTful standards.
Postman	Used to test and validate all REST API endpoints by sending HTTP requests and verifying JSON responses using OAuth 2.0 authentication.